



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▾

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Fundamentals

What are Containers

Hello and welcome to this lesson providing an overview of Kubernetes—a powerful container orchestration platform originally developed by Google and now maintained as an open source project. Kubernetes (or K8s) has revolutionized container management, enabling developers and operations teams to deploy and scale applications with ease.

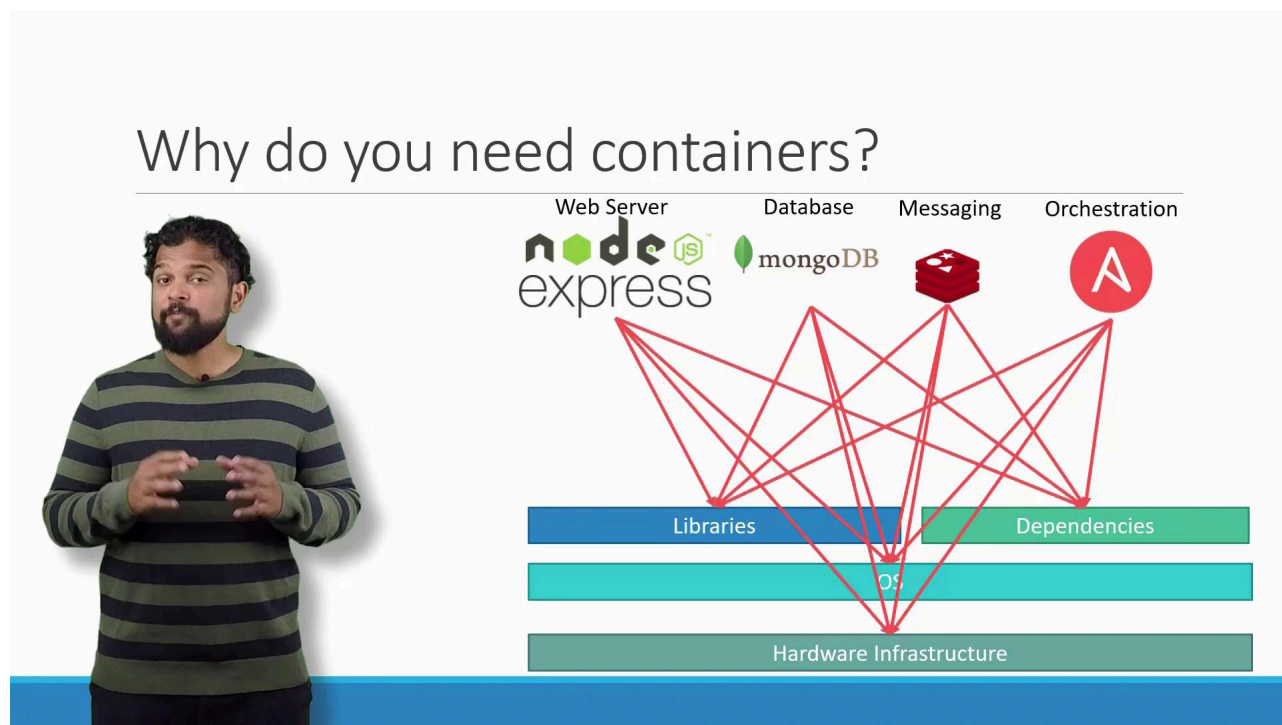
In this guide, we begin by examining two essential concepts: containers and orchestration. Understanding these fundamentals will set the stage for appreciating the capabilities of Kubernetes. Let's start by exploring containers with a focus on Docker, the most widely used container technology.

Introduction to Docker and Containers

If you are already familiar with Docker, feel free to skip ahead. However, I'd like to share an experience that highlights why Docker became vital in modern application development.

In one project, I faced the challenge of establishing an end-to-end application stack comprising multiple components: a Node.js web server, a MongoDB database, a Redis messaging system, and an orchestration tool such as Ansible. We encountered several compatibility issues:

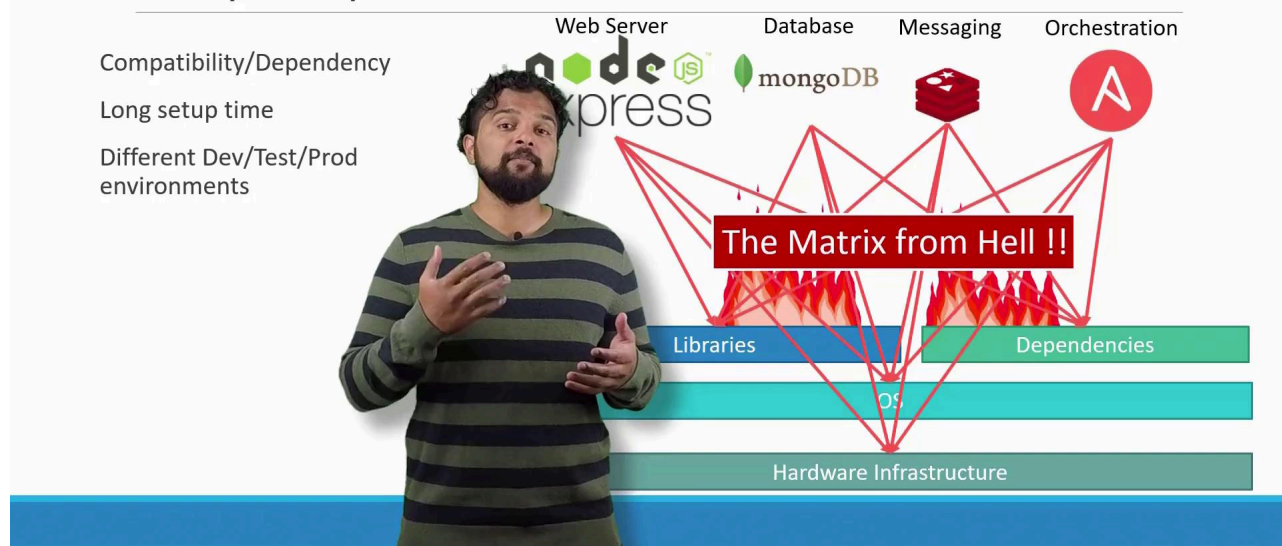
1. Services required specific operating system versions. Sometimes, mismatched versions meant that one service would work on an OS that another could not.
2. Services depended on different library versions, making it difficult to find a single environment that satisfied all dependencies.



Moreover, as the application evolved with upgrades and even component switches (like changing databases), each change brought new compatibility challenges, often described as the “matrix from hell.”

Another significant hurdle was onboarding new developers. They had to execute a long list of manual setup instructions, ensuring that their development environments matched production exactly—resulting in inconsistencies and higher error rates.

Why do you need containers?



Note

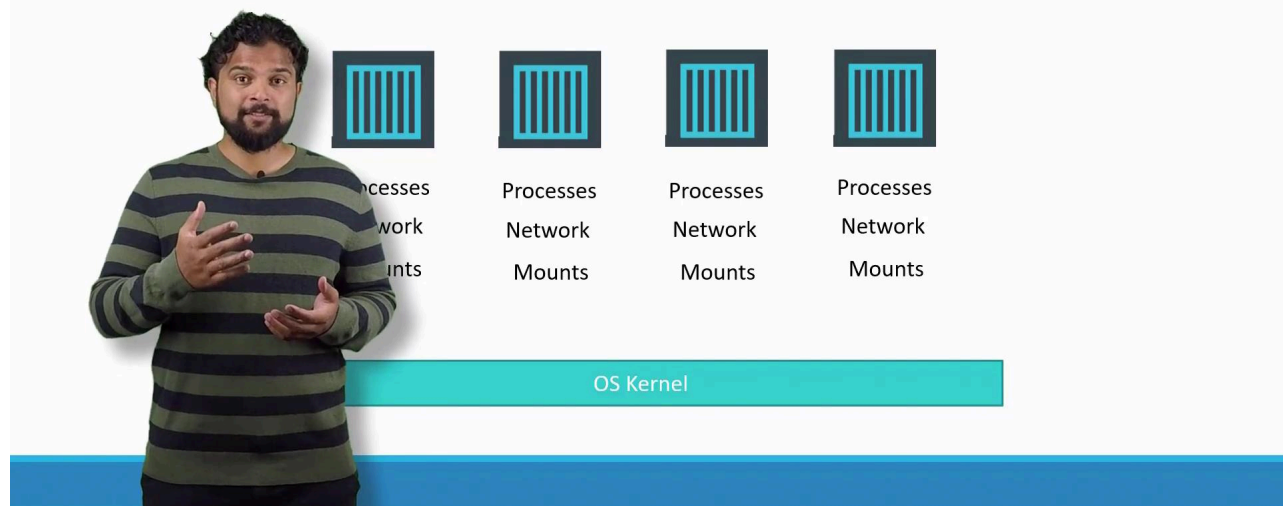
Docker solved these challenges by encapsulating each application component in its own isolated container. This isolation bundled libraries, dependencies, and a lightweight file system within each container—enabling seamless environment replication across development, testing, and production.

With Docker, developers simply run a Docker command as long as Docker is installed, regardless of the underlying operating system.

What Are Containers?

Containers provide isolated environments that run processes, services, network interfaces, and file system mounts—similar to virtual machines (VMs). However, unlike VMs, containers share the host's operating system kernel.

What are containers?



The concept of containerization is not new. Early implementations such as LXC, LXD, and LXCFS paved the way for Docker, which popularized containers by offering an easy-to-use, high-level interface that abstracts the complexities of container management.

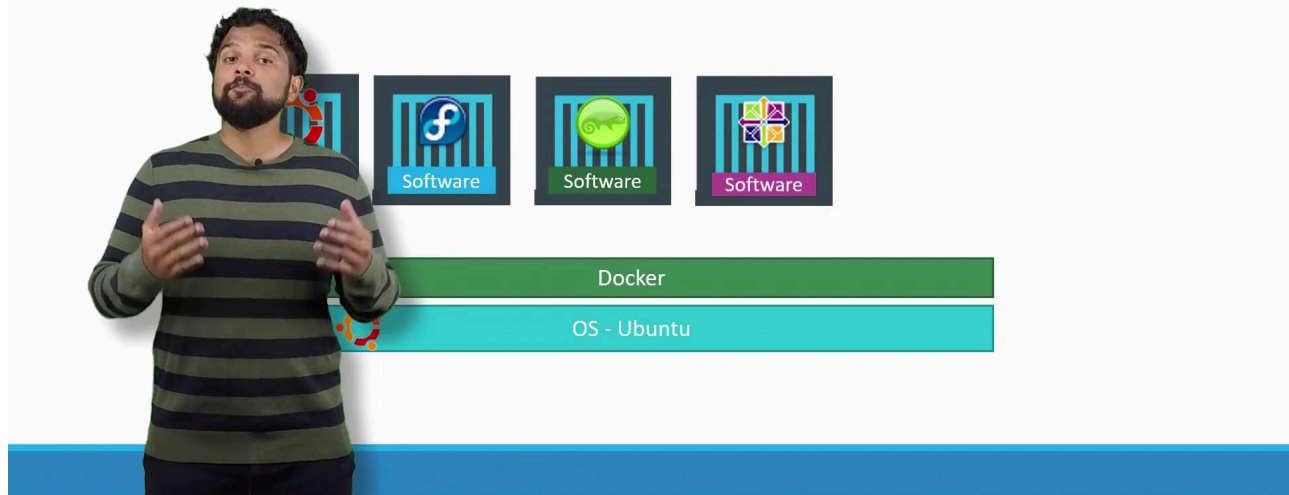
The Role of the OS Kernel

Understanding Docker's power starts with the operating system. Consider popular Linux distributions like Ubuntu, Fedora, SUSE, or CentOS. Each consists of:

- The OS Kernel: Interacting directly with hardware.
- A set of software packages: These define the user interface, drivers, compilers, file managers, and developer tools.

Docker containers share the host's kernel. For instance, an Ubuntu system running Docker can host containers based on Debian, Fedora, SUSE, or CentOS, as all share the same kernel. Each container includes only the additional software that sets it apart.

Sharing the kernel

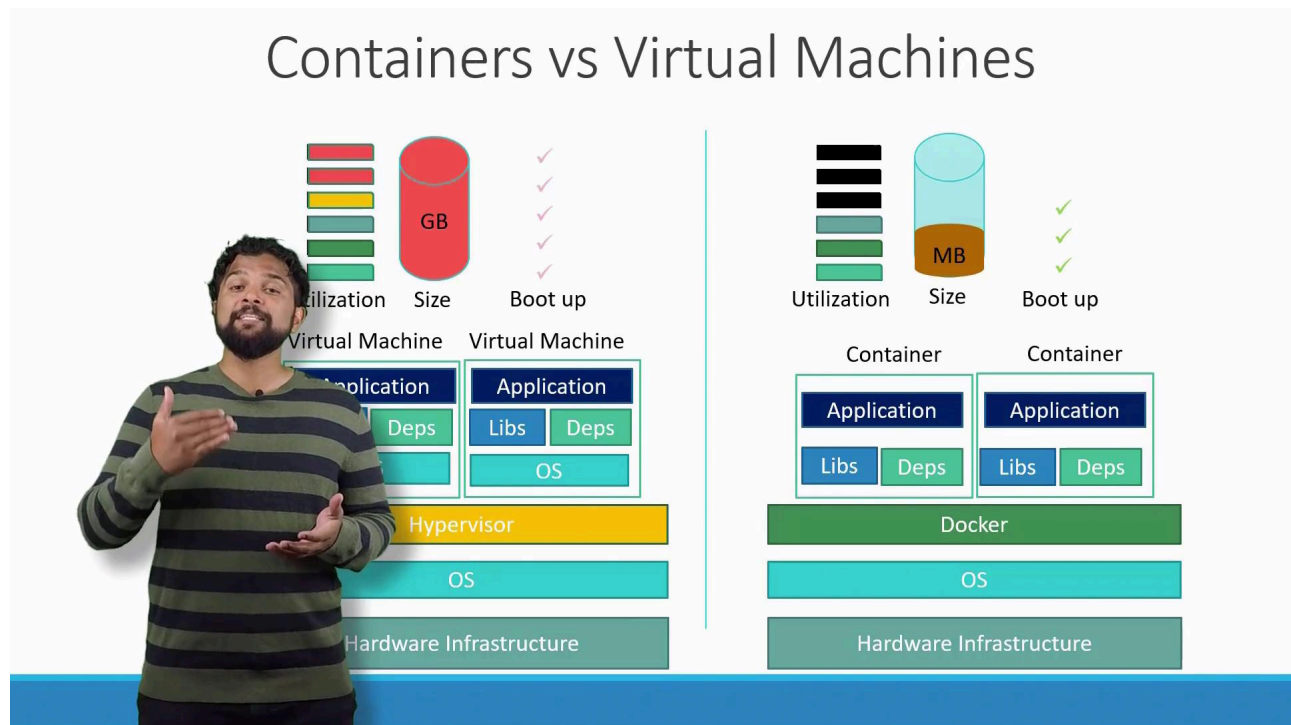


A key exception is Windows. Docker on Linux cannot run Windows-based containers due to kernel differences; for Windows containers, Docker must run on a Windows Server.

Containers vs. Virtual Machines

Understanding the distinction between containers and virtual machines (VMs) is crucial. Below is a comparison that highlights key differences:

Component	Docker Container	Virtual Machine
Base Architecture	Host OS + Docker daemon	Host OS + Hypervisor + Guest OS
Resource Consumption	Lightweight (measured in megabytes)	Typically larger (measured in gigabytes)
Boot Time	Seconds	Minutes
Isolation Level	Shares host kernel	Full OS isolation



While containers require less overhead, they provide a lower level of isolation compared to VMs since they share the host's kernel.

Running Containerized Applications

Many organizations today leverage containerized applications available on public registries like Docker Hub or Docker Store. These registries offer a vast range of images—from operating systems to databases and development tools.

Once you have the necessary Docker images and Docker is installed, deploying an application stack can be as simple as executing a series of commands. For example, you might launch services with these commands:

```
docker run ansible
docker run mongodb
docker run redis
docker run nodejs
```

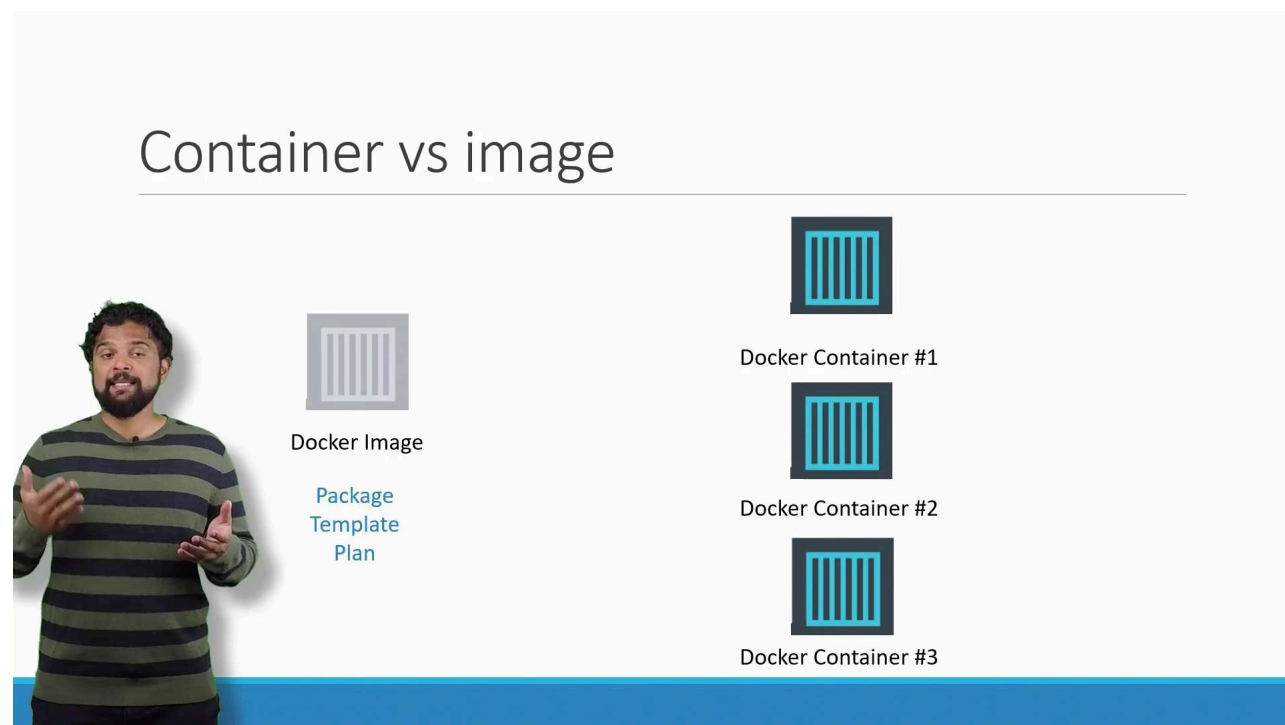
If you need multiple instances of a web service, you can run additional containers and configure a load balancer to distribute traffic. Should one instance fail, it can be

replaced quickly with minimal effort.

Understanding Images and Containers

It is important to differentiate between Docker images and containers:

- A Docker image is a template containing instructions to create a container—similar to a VM template.
- A Docker container is a running instance of that image, providing an isolated environment with its own processes and resources.



Many popular products are “Dockerized” already. If a prebuilt image does not meet your requirements, you can create one using a Dockerfile and push it to Docker Hub for wider use.

Shifting Responsibilities: Developers and Operations

Traditionally, developers built applications and handed them off to the operations team for deployment. The operations team then configured the host environment,

installed prerequisites, and managed dependencies. This handoff often led to miscommunications and troubleshooting difficulties.

Docker transforms this dynamic. Developers now define the application environment within a Dockerfile. Once the Docker image is built and tested, it behaves consistently across all platforms, simplifying the deployment process for operations teams.

Container Advantage



Developer



Docker Image



Operations



Key Takeaway

By encapsulating the application environment in a Docker image, you eliminate the "works on my machine" problem. The image, once verified in development, guarantees consistent production behavior.

With this foundation in containers and Docker, you are now ready to explore Kubernetes and harness its powerful orchestration capabilities.



Watch Video

Watch video content

Previous

◀ **Course Introduction**

Next

Demo Docker ▶